

Iterator Design

A complete, progressive interview guide to the Iterator design pattern in Java — custom Iterable/Iterator implementations, filtering, limiting, and composing iterators, lazy evaluation, fail-fast behavior, SOLID design, and a full senior-level interview question bank.

1. Why do we need Iterator?	[01]
2. Iterable vs Iterator	[02]
3. ZigZag Iterator	[03]
4. Odd Iterator	[04]
5. Filtering Iterator	[05]
6. Interleaving Iterator	[06]
7. Fail-Fast Iterator	[07]
- ModCount Concept	

Start With a Simple for Loop

```
for (int i=0; i<notifications.length; i++) {  
    notification = notifications[i]  
}
```

```
Iterator itr = notificationList.createIterator();  
while (itr.hasNext()) {  
    notification = itr.next();  
}
```

Why Do We Need an Iterator?

Suppose we have:

```
class UserCollection {  
    private User[] users;  
}
```

One simple design would expose:

```
public User[] getUsers() {  
    return users;  
}
```

The client can then do:

```
User[] users = Collection.getUsers();  
  
for (User user : users) {  
    // ...  
}
```

Ask:

What problems do you see?

Problem 1 — Internal Representation Leaks

The client knows the collection is implemented using:
`User[]`

If we change it to:

`List<User>`

the client may need to change.

Problem 2 — Tight coupling

The client is coupled to the storage structure.

Problem 3 — Different Traversal Strategies

Suppose we want:

Forward	Negative only
Backward	ZigZag
Odd only	Limited
Even only	Filtered

We don't want the collection API to expose every possible traversal implementation.

What Happens Inside Enhanced for?

Conceptually:

```
for (Integer number : numbers) {  
    System.out.println(number);  
}
```

works like:

```
Iterator<Integer> iterator = numbers.iterator();  
  
while (iterator.hasNext()) {  
    Integer number = iterator.next();  
    System.out.println(number);  
}
```

for-each

```
|  
v  
Iterable  
|  
| iterator()  
v  
Iterator  
|  
+---- hasNext()  
|  
+---- next()
```

Iterable vs Iterator

<pre>interface MyIterator<T> { boolean hasNext(); T next(); }</pre>	<pre>Interface Aggregator<T> { MyIterator<T> iterator(); }</pre> or <pre>Interface Iterable<T> { MyIterator<T> iterator(); }</pre>	Ex: <pre>Iterator<Integer> iterator = numbers.iterator();</pre> The iterator owns traversal state. For example: current index = 5
--	---	---

An Iterable<T> represents: Something from which an iterator can be obtained.

Implement a Basic Iterator

<pre>class UserIterator implements MyIterator<User> { private final User[] users; private int currIdx = 0; // constructor public UserIterator(User[] users) { this.users = users; } @Override boolean hasNext() { return currIdx < users.length; } @Override Employee next() { if(!hasNext()) throw new NoSuchElementException(); return users[currIdx++]; } }</pre>	<pre>class UserCollection { private final User[] users; public UserCollection(User[] users) { this.users = users; } @Override MyIterator<User> iterator() { return new UserIterator(users); } }</pre>
<pre>MyIterator<User> iterator = collection.iterator(); while (iterator.hasNext()) { User user = iterator.next(); System.out.println(user); }</pre>	

ZigzagIterator

Given a 2D array, output the results in interleaving fashion

EX; [[1, 2, 3], [4, 5], [6], [], [7, 8, 9]]

Output: [1, 4, 6, 7, 2, 5, 8, 3, 9]

```
// using Dequeue:
class ZigzagIterator {
    List<List<Integer>> combinedList;
    Dequeue<Pair> dq;

    ZigzagIterator(List<Integer> l1, List<Integer> l2) {
        combinedList = new ArrayList<>();
        dq = new ArrayDeque<>();

        combinedList.add(l1);
        combinedList.add(l2);

        for(int i=0; i< combinedList.size(); i++) {
            if(combinedList.get(i).size() > 0) {
                dq.addLast(new Pair(i, 0));
            }
        }
    }

    int next() {
        Pair pair = dq.pollFirst();
        List<Integer> currList = combinedList.get(pair.index);
        int res = currList.get(pair.offset);

        if(pair.offset + 1 < currList.size()) {
            dq.addLast(new Pair(pair.index, pair.offset + 1));
        }
        return res;
    }

    boolean hasNext() {
        return dq.size() > 0;
    }
}
```

```
// without using Dequeue:
class ZigzagIterator {
    List<List<Integer>> combinedList;
    Queue<Iterator<Integer>> itrq;

    ZigzagIterator(List<List<Integer>> lists) {
        itrq = new LinkedList<>();
        for(List<Integer> list : lists) {
            if(!list.isEmpty()) {
                itrq.add(list.iterator());
            }
        }
    }

    int next() {
        Iterator<Integer> currItr = itrq.poll();
        int res = currItr.next();

        if(currItr.hasNext()) {
            itrq.add(currItr);
        }
        return res;
    }

    boolean hasNext() {
        return !itrq.isEmpty();
    }
}
```

Oddlterator

EX; [1,2,3,4,5,6,7,8,9,10]; Output: [1, 3, 5, 7, 9]

```
class Oddlterator implements Mylterator<Integer>{
    Int curr;

    Oddlterator() {this.curr = 1; }

    boolean hasNext() {return true; }

    int next() {
        Int nextOdd = curr;
        Curr += 2;
        Return nextodd;
    }
}
```

```
class Evenlterator implements Mylterator<Integer>{
    final Oddlterator oddltr;

    Oddlterator(Oddlterator itr){... }
    boolean hasNext() {return true; }
    int next() {return oddltr.next(); }
}
```

Example:
var evenltr = new Evenlterator(oddltr);
S.O.P("First 5 even numbers: ");
for(int i=0; i< 5; i++) S.O.P(evenltr.next());

```
class Oddlterator implements Iterator<Integer>{
    final Iterator<Integer> source;
    private Integer nextValue;
    private boolean prepared;

    Oddlterator(Iterator<Integer> source){
        this.source = source;
    }

    private void prepare() {
        if(prepared) return;
        while (source.hasNext()) {
            Int value = source.next();
            if(value %2 == 0) {
                nextValue = value;
                prepared = true;
                return;
            }
        }
        prepared = true;
        nextValue = null;
    }

    boolean hasNext() {
        prepare();
        return nextValue != null;
    }

    int next() {
        prepare();
        if (nextValue == null)
            throw new NoSuchElementException();
        Integer result = nextValue;
        nextValue = null;
        prepared = false;
        return result;
    }
}
```

OddEven Iterator Even Iterator using Odd Iterator

Negativelterator

I/P: [10 -5 3 -8 -2 7] → [-5 -8 -2]

Positivelterator

I/P: [10 -5 3 -8 -2 7] → [10 3 7]

Rangelterator

Rangelterator(5, 10) → [5 6 7 8 9 10]

RangelteratorWithStep

Rangelterator(0, 10, 2) → [0 2 4 6 8 10]

Skiplterator

[1 2 3 4 5 6 7] → [4 5 6 7]

Limitlterator

[1 2 3 4 5 6 7]; limit=3 → [1 2 3]

FilteringIterator

Are OddIterator, EvenIterator, NegativeIterator and PositiveIterator duplicating the same algorithm?

Yes — The only thing changing is the condition.

```
class FilteringIterator<T> implements Iterator<T> {
    private final MyIterator<T> source;
    private final Predicate<T> predicate;
    private T nextValue;
    private boolean prepared;

    FilteringIterator(
        Iterator<T> source,
        Predicate<T> predicate) {
        this.source = source;
        this.predicate = predicate;
    }

    private void prepare() {
        if (prepared) {
            return;
        }

        while (source.hasNext()) {
            T candidate = source.next();
            if (predicate.test(candidate)) {
                nextValue = candidate;
                prepared = true;
                return;
            }
        }
        nextValue = null;
        prepared = true;
    }

    @Override
    public boolean hasNext() {
        prepare();
        return nextValue != null;
    }

    @Override
    public T next() {
        prepare();
        if (nextValue == null) {
            throw new NoSuchElementException();
        }

        T result = nextValue;

        nextValue = null;
        prepared = false;

        return result;
    }
}
```

Odd:

```
new FilteringIterator<>(
    iterator,
    value -> value % 2 != 0
);
```

Negative:

```
new FilteringIterator<>(
    iterator,
    value -> value < 0
);
```

ForwardIterator:

[1 2 3 4 5] → [1 2 3 4 5]

```
next():
    return data[index]
    index++
```

BackwardIterator:

[1 2 3 4 5] → [5 4 3 2 1]

```
int currIdx = list.size() - 1
hasNext():
    return index >= 0;
next():
    return list.get(index--);
```

Can BackwardIterator Work With Iterator<T>?

Can you implement BackwardIterator if the only input is: Iterator<T>

No — A normal iterator provides:

hasNext() and next()

It does not provide: **previous()**

DistinctIterator:

[1 2 2 3 3 3 4] → [1 2 3 4]

```
Set<T> seen = new HashSet<>();
```

InterleavingIterator

Given: A = 1 2 3 4 B = 10 20 30 40 Output: [1 10 2 20 3 30 4 40]	Architecture: <pre>Iterator A -----\ \ / InterleavingIterator / Iterator B -----/</pre>
---	---

Algorithm: if both have values: alternate if A is exhausted: continue B if B is exhausted: continue A	RoundRobinIterator Given: A = 1 2 3 B = 10 20 C = 100 200 300 Output: [1 10 100 2 20 200 3 300]
---	---

Zipliterator A = 1 2 3 B = 10 20 30 Return: (1,10) (2,20) (3,30) Conceptually: A.next() + B.next()	CycleIterator A = 1 2 3 → 1 2 3 1 2 3 1 2 3 ...] Can you implement CycleIterator using only Iterator<T> and without caching? No — A normal iterator eventually reaches: 1 -> 2 -> 3 -> exhausted There is no generic: restart() or previous() Therefore, without caching, the source must provide another way to restart traversal
---	--

remove()

Java's Iterator historically supports: boolean hasNext(); T next(); void remove();	What should happen if remove() is called before next() ? Usually invalid. Ask: What if remove() is called twice without another next()? Also invalid. Therefore an iterator supporting remove() has a richer state machine.
--	---

Fail-Fast Iterator

remove() State Machine

START next () v CAN_REMOVE remove () v CANNOT_REMOVE next () v CAN_REMOVE	<pre>List<Integer> list = new ArrayList<>(List.of(1, 2, 3)); Iterator<Integer> iterator = list.iterator();</pre> Then the list is modified: <code>list.add(100);</code> while iteration is in progress. What should happen? Java collection iterators often use fail-fast behavior and may throw: ConcurrentModificationException
---	---

modCount Concept

Conceptually: <pre>class Collection { int modCount; }</pre>	Iterator checks:
Iterator remembers:	<pre>if (expectedModCount != modCount) { throw new ConcurrentModificationException(); }</pre>
When the collection changes structurally:	Important:
<pre>modCount++</pre>	Fail-fast behavior is a detection mechanism, not a thread-safety mechanism.